ARIADNA NÚÑEZ TORRES

2 de septiembre de 2026

Conceptos básicos de Regresión Logistica, Ecuación base y Aplicación a modelo simple de Predicción de Ventas

Observacion y aplicacion de tecnicas de entrenamiento de evaluacion y visualización de resultados de la regresión logistica

Importacion de datos y librerias

```
# Importar libreriras a utilizar
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import accuracy_score, recall_score, f1_score, precision_score
from sklearn.linear_model import LogisticRegression
```

Se importan as principales librerías a utilizar, empezando por pandas para generar dataframes, numpy para arreglos que serán necesarios en ciertas partes del proceso, así como módulos o secciones específicas de Sklearn para la construcción del modelo, métricas, regresiones y demás…

```
# Importar archivo csv a utilizar
data = pd.read_csv('databses/general_data_.csv')
data
```

✓ 1.7s

	Age	Attrition	BusinessTravel	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeID	Gender
0	51	No	Travel_Rarely	Sales	6	2	Life Sciences	1	1	Female
1	31	Yes	Travel_Frequently	Research & Development	10	1	Life Sciences	1	2	Female
2	32	No	Travel_Frequently	Research & Development	17	4	Other	1	3	Male
3	38	No	Non-Travel	Research & Development	2	5	Life Sciences	1	4	Male
4	32	No	Travel_Rarely	Research & Development	10	1	Medical	1	5	Male

Se importa el archivo csv a utilizar generando un dataframe con este

Exploratory Data Analysis

```
#Obtencion de los primeros resultados del DataFrame
data.head(5)
```

✓ 0.0s

	Age	Attrition	BusinessTravel	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeID	Gender
0	51	No	Travel_Rarely	Sales	6	2	Life Sciences	1	1	Female
1	31	Yes	Travel_Frequently	Research & Development	10	1	Life Sciences	1	2	Female
2	32	No	Travel_Frequently	Research & Development	17	4	Other	1	3	Male
3	38	No	Non-Travel	Research & Development	2	5	Life Sciences	1	4	Male

```
#Obtencion de los ultimos resultados del DataFrame
data.tail(5)
```

✓ 0.0s

	Age	Attrition	BusinessTravel	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeID	Gende
4405	42	No	Travel_Rarely	Research & Development	5	4	Medical	1	4406	Fema
4406	29	No	Travel_Rarely	Research & Development	2	4	Medical	1	4407	Ma
4407	25	No	Travel_Rarely	Research & Development	25	2	Life Sciences	1	4408	Ma

Se empieza el EDA con seleccionar los primero y últimos datos del DataFrame, tal como obtener muestras aleatorias de esta, todo con el fin de tener una observación generalizada de la base datos con la que se trabajara y manipulara para la generación de del modelo

```
#Obtencion de muestras del DataFrame
data.sample(5)
```

✓ 0.0s

	Age	Attrition	BusinessTravel	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeID	Gender
1207	36	No	Travel_Frequently	Research & Development	5	1	Technical Degree	1	1208	Male
328	24	No	Non-Travel	Sales	1	5	Marketing	1	329	Female
2560	40	No	Travel_Rarely	Research & Development	3	3	Other	1	2561	Male
4109	40	No	Travel_Rarely	Research & Development	1	4	Medical	1	4110	Male
3477	37	No	Travel_Rarely	Research & Development	7	4	Life Sciences	1	3478	Male

ARIADNA NÚÑEZ TORRES

2 de septiembre de 2026

```
#Obtener la estadistica básica del Dataframe
data.describe().T
```

✓ 0.0s

	count	mean	std	min	25%	50%	75%	max
Age	4410.0	36.923810	9.133301	18.0	30.00	36.0	43.00	60.0
DistanceFromHome	4410.0	9.192517	8.105026	1.0	2.00	7.0	14.00	29.0
Education	4410.0	2.912925	1.023933	1.0	2.00	3.0	4.00	5.0
EmployeeCount	4410.0	1.000000	0.000000	1.0	1.00	1.0	1.00	1.0
EmployeeID	4410.0	2205.500000	1273.201673	1.0	1103.25	2205.5	3307.75	4410.0
JobLevel	4410.0	2.063946	1.106689	1.0	1.00	2.0	3.00	5.0
MonthlyIncome	4410.0	65029.312925	47068.888559	10090.0	29110.00	49190.0	83800.00	199990.0
NumCompaniesWorked	4391.0	2.694830	2.498887	0.0	1.00	2.0	4.00	9.0
PercentSalaryHike	4410.0	15.209524	3.659108	11.0	12.00	14.0	18.00	25.0
StandardHours	4410.0	8.000000	0.000000	8.0	8.00	8.0	8.00	8.0

Se obtiene las estadísticas básicas de los datos notando sus especificaciones de ciertos detalles estadísticos, pero lo que principalmente llama la atención es que este mismo tiene errores faltantes

```
#Identificar si hay columnas con nulos
missing_values = data.isnull().sum().sum()

print(f'Existen', missing_values, 'valores nulos')
```

✓ 0.0s

```
Existen 28 valores nulos
```

```
#Identificar las columnas especificas con dichos valores
missing_values_col = data.isnull().sum()
data.columns[missing_values_col.to_numpy().nonzero()]
```

✓ 0.0s

```
Index(['NumCompaniesWorked', 'TotalWorkingYears'], dtype='object')
```

Próximo a eso lo que se realiza es un análisis particular a esta parte identificando cuantos valores nulos en total son los que se encuentran y de que columnas son específicamente.

```
#Saber el tipo de dato que contiene cada columna
data.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4410 entries, 0 to 4409
Data columns (total 24 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Age                      4410 non-null   int64
 1   Attrition                4410 non-null   object
 2   BusinessTravel           4410 non-null   object
 3   Department               4410 non-null   object
 4   DistanceFromHome         4410 non-null   int64
 5   Education                4410 non-null   int64
 6   EducationField           4410 non-null   object
 7   EmployeeCount            4410 non-null   int64
 8   EmployeeID               4410 non-null   int64
 9   Gender                   4410 non-null   object
 10  JobLevel                 4410 non-null   int64
 11  JobRole                  4410 non-null   object
 12  MaritalStatus            4410 non-null   object
 13  MonthlyIncome            4410 non-null   int64
 14  NumCompaniesWorked       4391 non-null   float64
 15  Over18                   4410 non-null   object
 16  PercentSalaryHike        4410 non-null   int64
```

Como siguiente paso a realizar es sobre saber el tipo de dato, la mayoría serán ***object***, en su mayoría serán convertidas a categoría a excepción a las que serán eliminadas en un parte del proceso

```
#Ubicar si la mayoria de columnas cuentan con datos unicos
data.nunique()
```

✓ 0.0s

```
Age                        43
Attrition                   2
BusinessTravel              3
Department                  3
DistanceFromHome           29
Education                   5
EducationField              6
EmployeeCount               1
EmployeeID               4410
Gender                      2
JobLevel                    5
JobRole                     9
MaritalStatus               3
MonthlyIncome            1349
NumCompaniesWorked         10
Over18                      1
PercentSalaryHike          15
StandardHours               1
StockOptionLevel            4
```

Antes de pasar a ***Feature Engineering*** para realizar las siguientes trasformaciones y eliminaciones lo que se hará será identificar los datos que no aporten y además contengan un dato único, en este caso será ***EmployeeCount, Over18*** & ***StandardHours***

Imputación de datos

```
#Se obtiene el promedio general
#Calculando el promedio de ambas varaiables con nulos
avg_numcompanieswork = int(data['NumCompaniesWorked'].mean())
num_total_working_years = int(data['TotalWorkingYears'].mean())

#Se realiza la imputacion de promedio a todos los registros nulos de estas columnas
data['NumCompaniesWorked'] = data['NumCompaniesWorked'].fillna(avg_numcompanieswork)
data['TotalWorkingYears'] = data['TotalWorkingYears'].fillna(num_total_working_years
```

✓ 0.0s

```
#Identificar si hay columnas con nulos
missing_values = data.isnull().sum().sum()

print(f'Existen', missing_values, 'valores nulos')
```

✓ 0.0s

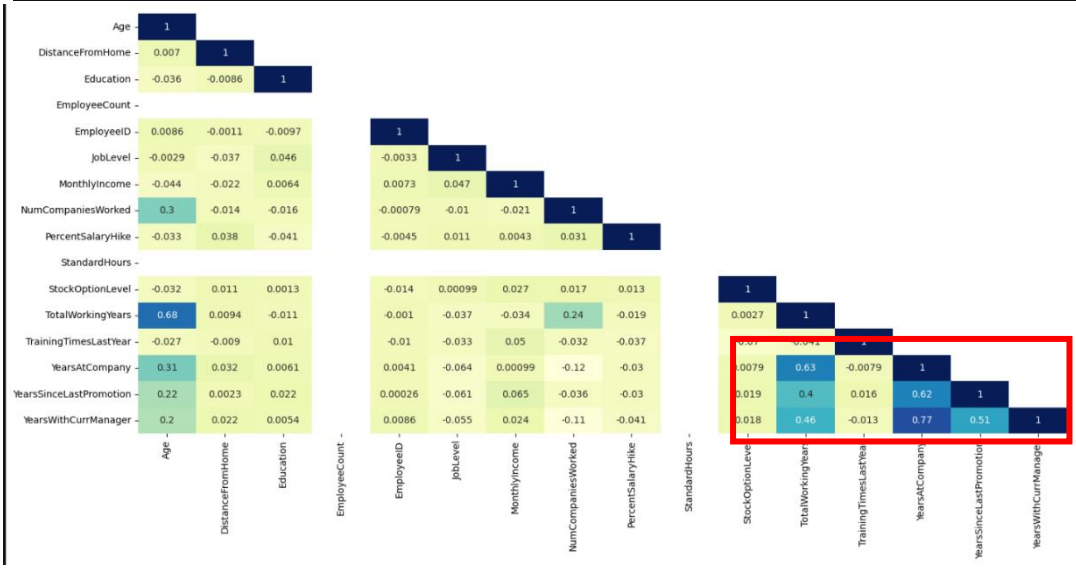
```
Existen 0 valores nulos
```

También otro punto antes de pasar a esa sección lo que se hará será una ***imputación de datos*** por ***promedio general*** remplazando todos los nulos por datos que serán calculados desde antes para poder ocupar esos espacios que podrían causar baja precisión y calidad en el modelo

ARIADNA NÚÑEZ TORRES 2 de septiembre de 2026

Análisis de correlación

```
#Se realiza un mapa de calor para saber si hay variables con correlacion alta
plt.figure(figsize =(22,8))
corr_data = corr = data.corr(method = 'pearson', numeric_only= True)
data_lt = corr_data.where(np.tril(np.ones(corr_data.shape)).astype(bool))
hmap = sns.heatmap(data_lt,cmap = 'YlGnBu',annot = True)
```



¿Por qué se realiza un análisis de correlación que uno bivariado y por qué con forma de triangulo?

Por que se necesitan identificar columnas que tengan alta correlación que en dado de ser así pueda ocasionar entorpecimiento por hallazgos nuevos o relevantes de patrones por parte del modelo. Además, la forma de triangulo permite que los datos se agrupen y con una visibilidad mejorada que en comparación a la de cuadrado que divide los datos poco comprensibles y no permite localizar las variables con alta correlación

Feature Engineering

```
#Eliminacion de columnas a no utilizar
data.drop('EmployeeCount',axis=1, inplace = True)
data.drop('Over18',axis=1, inplace = True)
data.drop('StandardHours',axis= 1, inplace = True)
data.drop('EmployeeID',axis =1, inplace= True)

#Cambiar esta columna a tipo entero pero aparte llenar ese espacio por si tiene nulos
data['NumCompaniesWorked'] = data['NumCompaniesWorked'].fillna(0).astype(np.int64)
#Se imputa TotalWorkingYears como el promedio
num_total_working_years_mean = int(data['TotalWorkingYears'].mean())
#Imputar el promedio a todos los registros definidos como NaN
data['TotalWorkingYears'] = data['TotalWorkingYears'].fillna(num_total_working_years_mean)

#Transformación de tipo de dato
data = data.astype({
    'BusinessTravel': 'category',
    'Department': 'category',
    'EducationField':'category',
    'Gender':'category',
    'JobRole':'category',
    'MaritalStatus':'category'
})
```

Dentro de ***Feature Engineering*** lo que se realizará será la eliminación de columnas que no aportan y quitan precisión al modelo, además de modificar el tipo de dato a categórico en varias de las columnas así como a unas era a integro debido a que no puedes trabajar a mitades de una empresa, es decir que se cambio esta columna de ***float*** a **int**

Es importante que todas las columnas tengan su respectiva transformación de tipo de dato ya que en pasos posteriores pueda tener varios errores sobre todo con la regresión logística ya que no permite que haya valores flotantes (a excepción que sea la estandarización que se realizó)

```
#Category - codes - automatico
data.Gender = data.Gender.astype('category').cat.codes
data.EducationField = data.EducationField.astype('category').cat.codes
data.BusinessTravel = data.BusinessTravel.astype('category').cat.codes
data.Department = data.Department.astype('category').cat.codes
data.MaritalStatus = data.MaritalStatus.astype('category').cat.codes
data.JobRole = data.JobRole.astype('category').cat.codes
```

De igual manera se puede generar tipo código en base a su categoría para que no sea necesario realizar dummies

```
#Balance de clases
#Obtener distintos valores
print(data['EducationField'].value_counts())
print('')

print(data['BusinessTravel'].value_counts())
print('')

print(data['Department'].value_counts())
print('')
```

[23] ✓ 0.0s

```
EducationField
Life Sciences       1818
Medical             1392
Marketing            477
Technical Degree     396
Other                246
Human Resources       81
Name: count, dtype: int64

BusinessTravel
Travel_Rarely        3129
Travel_Frequently     831
Non-Travel            450
Name: count, dtype: int64
```

Se realiza un balance de clases como un reconteo por si es necesario tener que agrupar algunas columnas, convirtiendo ese grupo como "Otros" y que no sean datos incongruentes

```
#Cambiar manualmente la columna de 'attrition'
data['Attrition'] = data['Attrition'].replace({'Yes': 1,'No':0},regex = True)
```

✓ 0.0s

```
:\Users\USER\AppData\Local\Temp\ipykernel_8724\3336668630.py:2: FutureWarning:
data['Attrition'] = data['Attrition'].replace({'Yes': 1,'No':0},regex = True)
```

En esta sección la columna de ***Attrition*** se cambia manualmente a ser categórica para que pueda ser utiliza en procesos posteriores

Como siguiente acción para poder realizar la visualización de datos se estandarizará la columna de ***Monthly Income***

```
#Estandarizar la columna de Montly Income
scaler = StandardScaler()
data['MonthlyIncome'] = scaler.fit_transform(data[['MonthlyIncome']])
```

✓ 0.0s

ARIADNA NÚÑEZ TORRES

2 de septiembre de 2026

Data Split

- Entrenamiento, Evaluación y Prediccion de los resultados del modelo

```
#Se crea una copia del DataFrame original
data2 = data.copy()
```

✓ 0.0s

Se realiza una copia del dataframe principal para poder trabajar con ello al momento de construir el modelo

```
#Se crea primero el modelo en base a la columna a utilizar
x = data2.drop('Attrition', axis = 1)
y = data2['Attrition']

#Realizar el split de la data de 70% a 30%
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size= 0.30, random_state= 1)
#Checar que el dataframe se dividio de forma correcta
print('{0:0.2f}% la data de esta en un  set de training'.format((len(x_train)/len(data2.index))*100))
print('{0:0.2f}% la data de esta en un  set de training'.format((len(x_test)/len(data2.index))*100))
```

✓ 0.0s

```
70.00% la data de esta en un  set de training
30.00% la data de esta en un  set de training
```

Se comienza a seccionar el dataset en 70 de entrenamiento y 30 de testeo en próximos pasos notándose que esta vez si cumple con su 70% y 30% completamente

Modelo de Regresión Logistica

```
#Adapta el modelo de los x_train y y_train
model = LogisticRegression(solver = 'liblinear', class_weight='balanced')
model.fit(x_train,y_train)
#Hace la prediccion en x_test y la guarda en predict
y_predict = model.predict(x_test)
```

Se adaptan las variables de entrenamiento hacia el modelo de regresión logística permitiendo que puedan ser registrados correctamente para el modelo y con su respectiva predicción

```
#Se construye un dataset con el valor original y prediccion
z = x_test.copy()
z['Attrition real']= y_test
z['Attrition prediccion'] = y_predict
z[['Attrition real','Attrition prediccion']].head(10)
```

✓ 0.0s

	Attrition real	Attrition prediccion
2016	0	0
4118	0	1
219	0	0
4402	1	0
787	0	0
200	0	0
2539	0	0
3744	0	1
1891	0	1
3963	1	1

Al construir este data set se puede apreciar de lo que se puede tener como predicción y que en la matriz de confusión se puede representar al momento las predicciones

```
#Se genera el coeficiente de la ecuacion
coef_data = pd.DataFrame(model.coef_)
coef_data.columns = x.columns
#Incluyendo el intercept
coef_data['intercept'] = model.intercept_
coef_data.T
```

✓ 0.0s

	0
Age	-0.026643
BusinessTravel	0.018681
Department	-0.284441
DistanceFromHome	0.001540
Education	-0.040473
EducationField	-0.079893
Gender	0.124802
JobLevel	-0.073882
JobRole	0.052852
MaritalStatus	0.624619
MonthlyIncome	-0.067814
NumCompaniesWorked	0.134866
PercentSalaryHike	0.020809

Por lo que se puede observar varias de las variables van hacia un estado de negatividad esto debido al coeficiente de la ***ecuación***, así como el ***intercept*** a usar, esto se reflejara en pasos próximos de forma particular

Análisis de Sensibilidad

```
#Se construye un regresor lineal para este análisis
regression_model = LinearRegression()

#Se genera un fit de la regresion con las variables de x & y
regression_model.fit(x_train,y_train)
```

✓ 0.0s

LinearRegression

LinearRegression()

```
regression_model.score(x_train,y_train)
```

✓ 0.0s

```
0.09034124492301399
```

```
regression_model.score(x_test,y_test)
```

✓ 0.0s

```
0.08041279400543622
```

Para el análisis de sensibilidad se construye un regresor linear para realizar en pasos próximos lo que se requiere para un dataframe con las variables y su coeficiente.
Lo que se realiza en estas líneas de código es obtener el resultado de las variables de entrenamiento y testeo

ARIADNA NÚÑEZ TORRES

2 de septiembre de 2026

```
#Se genera un dataframe donde se muestre el coef y su respectiva variable
coef_data = pd.concat([coef_data, pd.DataFrame({'Coeff': [regression_model.intercept_], 'Var':['Intercept']})],ignore_index= True)
for idx, col_name in enumerate(x_train.columns):
    coef_data = pd.concat([coef_data, pd.DataFrame({'Var': [col_name], 'Coeff':[regression_model.coef_[idx]]})],ignore_index= True)

data_coef = coef_data.dropna(subset=['Coeff','Var']).reset_index(drop =True)
data_coef[['Coeff','Var']]
```

0.0s

	Coeff	Var
0	0.337762	Intercept
1	-0.004164	Age
2	-0.002299	BusinessTravel
3	-0.034782	Department
4	-0.000114	DistanceFromHome
5	-0.006084	Education
6	-0.010753	EducationField
7	0.016856	Gender
8	-0.003298	JobLevel

En esta sección lo que se hará será la observación de las variables con su debido coeficiente. Se seleccionan ***Coeff*** y ***Var*** para su principal detalle por parte de cada uno, esto se realiza por mediante los códigos unión por ***concat*** junto con la formación de lista por parte de ***for***

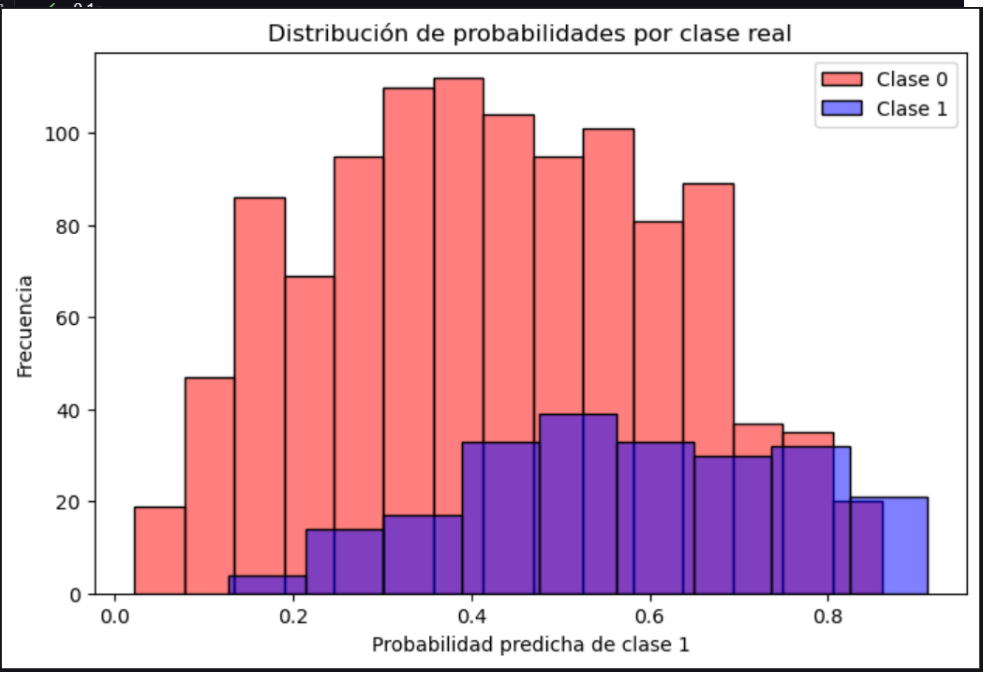
Visualización de datos del modelo de clasificación

Generate + Code + Markdown

```
#Se realiza un metodo para este tipo de modelos de clasificacion
#Devuelve la probabilidad
probs = model.predict_proba(x_test)[:, 1]
```

0.0s

```
#En vez de hacer un gráfico de scatter se realiza uno de barras
plt.figure(figsize=(8,5))
sns.histplot(probs[y_test == 0], color='red', label='Clase 0', alpha=0.5)
sns.histplot(probs[y_test == 1], color='blue', label='Clase 1', alpha=0.5)
plt.xlabel('Probabilidad predicha de clase 1')
plt.ylabel('Frecuencia')
plt.title('Distribución de probabilidades por clase real')
plt.legend()
plt.show()
```



Proba, un tipo de método especifico para modelos de clasificación y que de estos mismos se puedan usar dentro de la visualización de datos, la razón de esto es por que proba permite usar numeración binaria con base a las variables del modelo con x & y, para ello se utilizara un gráfico de barras ayudara a comparar los tipos de probabilidad que hay en 0 y 1

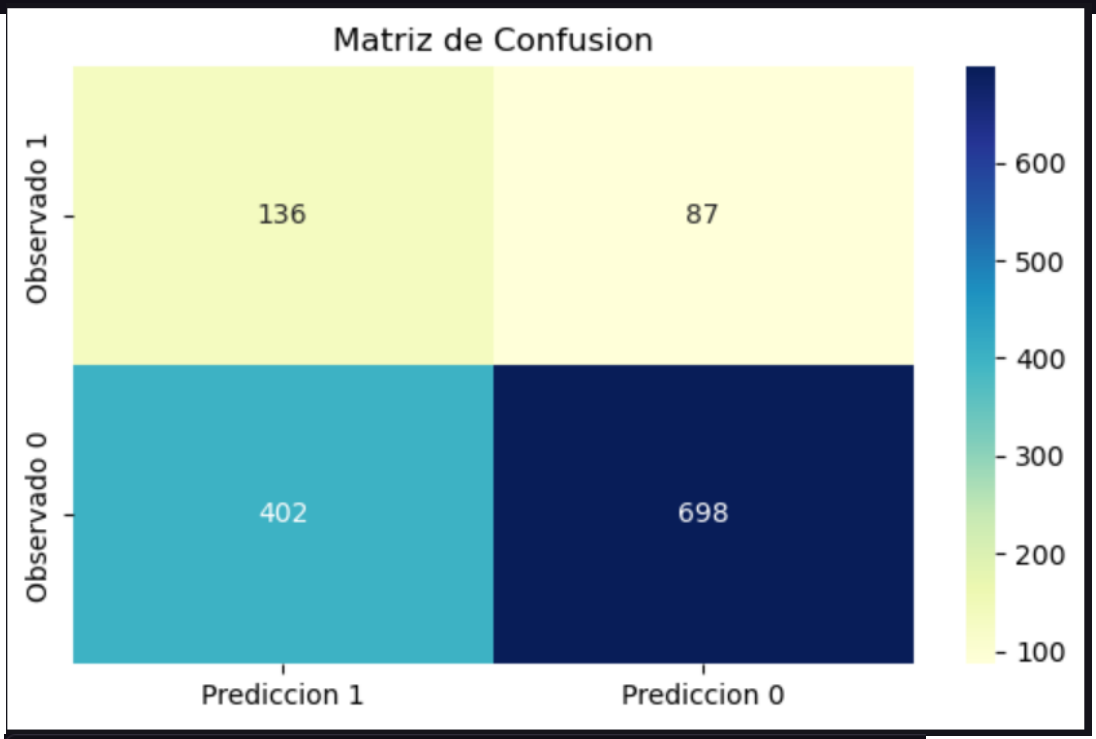
ARIADNA NÚÑEZ TORRES 2 de septiembre de 2026

Matriz de Confusión y metricas

```
cm = metrics.confusion_matrix(y_test,y_predict,labels = [1,0])

data_cm = pd.DataFrame(cm, index = [i for i in ['Observado 1', 'Observado 0']],
                       columns = [i for i in ['Prediccion 1', 'Prediccion 0']])

plt.figure(figsize= (7,4))
sns.heatmap(data_cm, annot = True, cmap = 'YlGnBu', fmt = 'd').set_title('Matriz de Confusion')
```



Lo que se obtuvo como resultado en la matriz de confusión es que el modelo predijo que los verdaderos positivos son en un 136 siendo una predicción alta, mientras que los falsos negativos se mantuvieron en 87, junto con 402 siendo falsos positivos o bien como falsas alarmas. Ahora con el digito mas alto de 698 de verdaderos negativos siendo si predicción mas alta y la mejor

```
#Medidas del modelo
y_pred = model.predict(x_test)
accuracy_score = accuracy_score(y_test,y_pred)
recall_score = recall_score(y_test,y_pred)
precision_score = precision_score(y_test,y_pred)
F1_score = f1_score(y_test,y_pred)

print('medidas del modelo')
print('---------------------------------')
print('Accuracy = {0:0.2f}'.format(accuracy_score))
print('Recall = {0:0.2f}'.format(recall_score))
print('Precision = {0:0.2f}'.format(precision_score))
print('F1 = {0:0.2f}'.format(F1_score))
```

✓ 0.0s

```
medidas del modelo
---------------------------------
Accuracy = 0.63
Recall = 0.61
Precision = 0.25
F1 = 0.36
```

Los resultados de las métricas por lo que mencionan es que en Accuracy tiene un 63% de aserción que va de la mano con usar recall a todos los verdaderos y falsos positivos en un 61%, aunque solo su precisión es bastante baja con su 25% dando una especificidad de 36% en negativos reales